Automatic Video Montage Generation Guided by Textual Prompts: A Multi-Modal Pipeline with Motion Segmentation, BLIP Captioning, and CLIP Matching

Luigi Daddario[1]

[1*]Artificial Intelligence for Science and Technology, University of Milano-Bicocca, Milan, Italy.

Contributing authors: l.daddario1@campus.unimib.it;

Abstract

This work presents a practical system for automatic video montage generation guided by short textual prompts. The goal is to select and assemble the most relevant moments from an input video according to user intent expressed in natural language. The proposed approach is a multi stage pipeline. It first detects motion segments through frame difference statistics. It then extracts representative frames and generates image captions using a BLIP model. The prompts are analysed with lightweight linguistic processing and optional semantic filtering based on sentence embeddings. Finally captions and prompts are matched with CLIP text embeddings and segments above a similarity threshold are selected and concatenated into the final montage. We report experiments that compare the proposed method against baselines that ignore semantics such as random selection uniform sampling first segments and motion intensity ranking. We also report an ablation on semantic filtering and a sensitivity analysis over similarity thresholds. Results on the provided test video show strong improvements over baselines and confirm that the two stage filtering strategy can reduce computation while keeping selection quality high.

Keywords: video summarization, video montage, vision-language models, BLIP, CLIP, motion segmentation, text prompts

1 Introduction

Video summarization is often treated as a task driven by simple features such as motion or shot boundaries. These features are useful but they do not capture user intent. In many scenarios the user already knows what they are searching for and can describe it with a short prompt. This motivates natural language guided montage generation where the summary is conditioned on text and not only on visual dynamics.

This project targets a simple but realistic setting. Given one input video and a list of prompts the system selects segments that best match the prompts and produces a montage. The focus is on a modular pipeline that can be run end to end and can also be analysed through experiments. The pipeline combines classical video processing with recent vision language models. Motion detection is used to narrow the search to dynamic regions. BLIP generates captions that describe the visual content. CLIP is used to align prompts and captions in a shared semantic space. An optional semantic filter based on sentence embeddings is used to reduce the number of captions to be scored by CLIP.

2 Objectives

The objectives of this work are the following.

- First we design an end to end pipeline that converts a raw video into a montage guided by user prompts.
- Second we aim for a system that is easy to reproduce and to evaluate. The code provides scripts for running the pipeline and for running comparative experiments that generate JSON results tables and figures.
- Third we study the contribution of individual components through baselines and ablations. We also study the effect of key thresholds to understand the precision recall trade off and the impact on coverage diversity and temporal coherence.

3 Methods

3.1 Dataset

We evaluate the system on the videos provided with the project repository. The main reference experiment uses a short cooking style video and three prompts that describe key actions. Motion segmentation produces a set of candidate temporal segments and each segment is represented by its center frame. The reported metrics and plots are generated by the experiment scripts and stored in the results folder.

3.1.1 Reference video, prompts, and transparency artifacts

For transparency and reproducibility, we report the exact reference configuration used throughout this paper. The input video is stored in the repository at data/videos/1.mp4 and the montage produced by the full pipeline is saved at outputs/test_montage.mp4. The pipeline is executed via run_complete_pipeline.py with the following prompts:

Table 1: Example captions generated by BLIP on the reference video and their maximum CLIP text–text similarity score (higher is more similar to at least one prompt).

Frame	Score	Caption
3357	0.867	a person is putting food into a box
998	0.856	a person is putting cheese on a sandwich
3174	0.849	a person is cutting up some food in a kitchen
3719	0.847	a person is putting something into a bowl
785	0.836	a person is putting a sandwich in a container
900	0.832	a person is putting food in a container

(a) Frame 3357 (b) Frame 998 (c) Frame 3174

(d) Frame 3719 (e) Frame 3473 (f) Frame 785

Fig. 1: Representative frames from the reference video corresponding to top-scored BLIP captions (see Table 1).

- *adding the ingredients in the sandwich*
- *closing the box*
- *plating the dish*

In addition to aggregate metrics, the run exports a machine-readable report containing the full list of generated captions, the subset that passes semantic filtering, and the CLIP similarity score used for selection (results/reference_example_report.json). Table 1 shows representative examples of BLIP captions and their maximum CLIP similarity score over the prompt set.

3.2 Optimization and Learning Setup

This work focuses on inference and matching rather than on training. The core models are used directly, without fine-tuning.

Caption generation uses BLIP for conditional captioning. In the current configuration we use the base captioning checkpoint from Salesforce. The generated captions are short and descriptive and they are suitable as intermediate textual representations.

Semantic matching uses CLIP. Instead of matching prompts directly to images we match prompts to captions. Both prompts and captions are embedded with the CLIP text encoder and cosine similarity is computed. For each caption we keep the maximum similarity over the set of prompts.

The pipeline includes two key thresholds. A motion threshold is computed as a percentile of per frame motion values and a similarity threshold selects segments for the final montage. We also optionally apply a semantic filter before CLIP; This filter uses sentence embeddings from a sentence transformer model and keeps only captions above a semantic similarity threshold. This step can reduce CLIP computations and can also remove clearly irrelevant captions.

Although our experimental results are obtained *without* any fine-tuning, we designed the repository to also support optional fine-tuning of the vision–language components as an extension of this project. For reproducibility and future work, we include an example implementation and instructions on how to run it (see README_FINETUNING.md and the example script at scripts/finetune_models.py).

3.3 Environment and State Representation

The system processes the video through a sequence of transformations. It first analyzes motion to identify temporal segments, selects a representative frame for each segment, and generates a caption describing its content. The user prompt is parsed into keywords and structures, and both captions and prompts are mapped into a shared embedding space. Similarity scores are computed using CLIP, and segments above a given threshold are selected. The corresponding video clips are then concatenated to form the final montage.

4 Experiments

4.1 Experimental Protocol

We follow a simple protocol. We first run the full pipeline with a fixed similarity threshold and we collect the selected segments. We then compare the proposed method to baselines that select the same number of segments. We also run an ablation where we disable the semantic filtering stage and, finally, we run a threshold sensitivity experiment where we recompute the selected segments for multiple similarity thresholds and evaluate the resulting montages.

All experiments are executed through the provided scripts. The main outputs are a JSON file that stores metrics and selected segments and a set of figures.

4.2 Baselines

To isolate the impact of *prompt semantics*, we compare against baselines that *do not use* BLIP/CLIP text alignment. Each baseline selects exactly N motion segments,

where N is the number of segments selected by the proposed method on the same run, to ensure a fair comparison in terms of montage length.

- *Random*: samples N motion segments uniformly at random.
- *Uniform*: samples N motion segments evenly across time (uniform temporal coverage).
- *First-N*: selects the first N motion segments in temporal order.
- *Motion-intensity*: ranks motion segments by average motion magnitude and selects the top N.

4.3 Implementation and Reproducibility

The project is implemented in Python and relies on standard libraries for vision and deep learning. Video processing uses OpenCV for reading frames and MoviePy for montage assembly. BLIP and the optional text summarization components use the Hugging Face Transformers ecosystem. CLIP uses the original OpenAI implementation. Prompt parsing optionally uses spaCy. Semantic filtering uses sentence transformers.

Reproducibility is supported through a fixed experimental setup, the prompts and the video path are specified in the command line scripts. Baseline methods that involve randomness use an explicit seed and the results folder stores the exact metric values used in this paper.

4.4 Results

This section summarizes the quantitative results produced by the experiment runner and relates them to the behaviour of the pipeline. We report metrics aimed at capturing both *relevance* (how well the selected clips match the user prompts) and *montage properties* (how much of the original content is retained and how coherent the final sequence is).

Because the dataset does not include dense, manually annotated ground truth for prompt relevance, traditional metrics such as precision, recall, and F1 are computed as *proxy* measures inside the pipeline rather than as ground-truth classification scores. Concretely, they measure *self-consistency* with the pipeline's own selection rule (CLIP-based similarity compared to a fixed threshold) instead of correctness with respect to human-labelled relevant/irrelevant segments.

4.4.1 What does "proxy" mean here?

In a standard evaluation setting, precision and recall require an external reference: a set of segments that are truly relevant to the prompts (positives) and truly irrelevant (negatives). In our setting, the only "labels" available are the pipeline decisions induced by the similarity threshold. Therefore, proxy precision/recall/F1 quantify whether the selected set behaves consistently with that internal decision boundary, and are primarily useful for *internal* comparisons across methods and ablations under the same protocol (e.g., proposed method vs. baselines constrained to select the same number of segments).

4.4.2 Why can proxy scores be 1.0?

Proxy precision/recall/F1 can saturate at 1.0 in stable regimes because the evaluation is not independent of the selection rule: if the selected set matches the thresholding behaviour for all candidate segments, there are no proxy false positives or proxy false negatives, yielding perfect scores even though this does not prove perfect semantic relevance to a human observer.

Alongside these proxy classification scores, we report *coverage*, which measures how much motion content from the source video is included in the montage, *diversity*, derived from caption vocabulary statistics (entropy-based), and *temporal coherence*, estimated from the average time gap between consecutive selected segments.

In the baseline comparison (Fig. 2 and Fig. 2), the proposed method consistently performs better than heuristics that ignore the prompt semantics. In the reference run, the full pipeline selects 43 segments and achieves proxy precision/recall/F1 of 1.0/1.0/1.0, with a coverage ratio of 0.496 and caption diversity of 0.728. Baselines that select an equal number of segments (random selection, uniform sampling, first segments, and motion-intensity ranking) obtain substantially lower proxy precision (around 0.581) and correspondingly lower F1 (around 0.735), while showing no consistent advantage in semantic relevance. Notably, motion-intensity ranking tends to increase coverage by favouring dynamic moments, but it does not reliably align selections with the textual intent expressed by the prompts.

4.4.3 The heatmap?

Figure 2 is a compact matrix view: each row is a method (proposed or baseline), each column is a metric, and the cell color encodes the metric value (darker/hotter colors indicate higher scores). This helps identify patterns at a glance, e.g., motion-intensity may improve coverage but does not improve semantic relevance.

We also quantify the contribution of the semantic filtering stage through an ablation study (Table 2 and Fig. 2). With semantic filtering enabled, the pipeline reduces the candidate set from 74 captions to 43, resulting in 43 selected segments. When semantic filtering is removed, all 74 candidates are processed and 74 segments are selected, leading to maximal coverage (1.0) but also to a significantly longer montage with lower selectivity. This highlights the practical role of the two-stage strategy: a lightweight semantic filter reduces computation and discourages clearly irrelevant candidates before CLIP-based scoring.

4.4.4 Ablation study?

An ablation study measures the contribution of a component by *removing it* while keeping everything else unchanged. Here we ablate semantic filtering to quantify its effect on how many candidates reach CLIP, montage length/coverage, and overall selectivity.

Finally, we run a sensitivity analysis over the CLIP similarity threshold in the range 0.15 to 0.45 (step 0.05, Fig. 2). On the reference video, the selected set is stable across the tested thresholds and the number of selected segments remains 43. This suggests a

Table 2: Ablation study on semantic filtering on video 1. Values are taken from the experiment outputs.

Configuration	Selected Segments	Coverage Ratio	Temporal Coverage
Full pipeline with semantic filtering	43	0.496	0.124
No semantic filtering	74	1.000	0.251

clear separation between the similarity scores of selected and non-selected candidates after semantic filtering, making the default threshold choice robust for this scenario.

4.4.5 Why does threshold sensitivity plot show perfectly flat lines?

There are two reasons. First, if the selected set does not change across thresholds, both the segment count and all downstream montage metrics remain constant. Second, precision/recall/F1 are computed as *proxy self-consistency* metrics with respect to the same selection rule used by the pipeline (they do not rely on external human annotations). In stable regimes, these proxy classification scores can saturate at 1.0, producing the near-perfect straight lines observed in the plot.

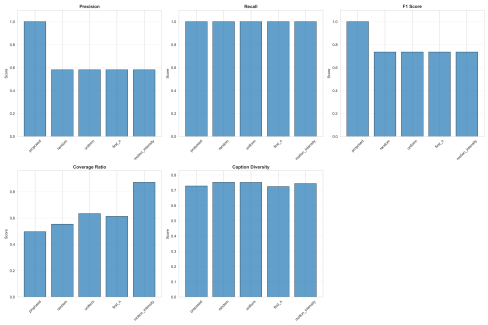
All numeric results (metrics and selected segments) are stored in the JSON outputs generated by the scripts, and the corresponding figures are provided in the *Visualizations* section (baseline comparison, metrics heatmap, ablation, and threshold sensitivity).

4.5 Discussion and Limitations

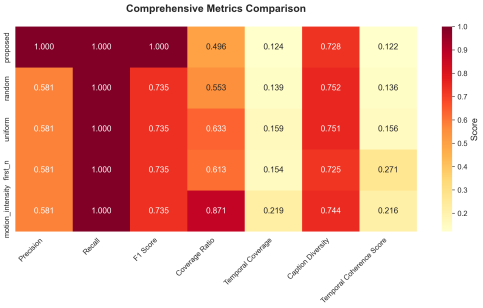
The experiments support the main intuition of the project. Semantics matter. Methods that ignore text and only rely on temporal heuristics or motion statistics can cover many segments but they are less selective and include many irrelevant moments. CLIP based matching improves relevance because it aligns prompt intent with caption meaning.

Semantic filtering before CLIP is conceptually useful because it reduces the candidate set that needs to be scored by CLIP. This can be important when the video contains many segments. The ablation study can quantify this impact on both computation and metrics once the ablation results are exported to figures and tables.

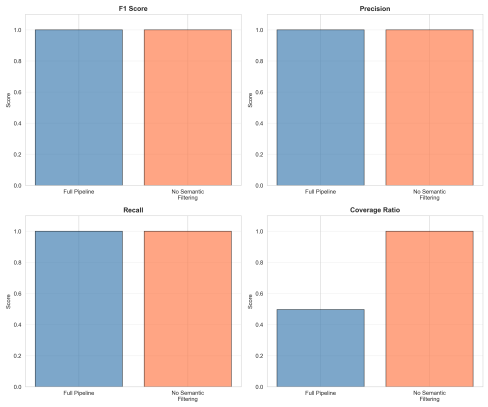
There are also limitations. The evaluation is based on proxy metrics because the system does not include manual annotation of prompt-relevant segments; this is adequate for internal comparison across methods, but it does not replace a human judgement study. Caption quality can also affect matching quality: if BLIP outputs overly generic descriptions, some prompt-specific actions may be missed. Finally, the montage uses hard cuts without transitions, which may reduce perceived coherence even when semantic relevance is high.



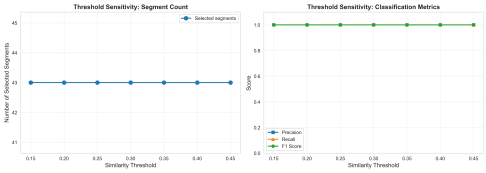
(a) Baseline comparison across multiple metrics. Baselines are constrained to select the same number of segments as the proposed method.



(b) Metrics heatmap. Rows are methods and columns are metrics; cell color encodes the metric value, enabling quick comparisons across methods.



(c) Ablation study: full pipeline vs. no semantic filtering.

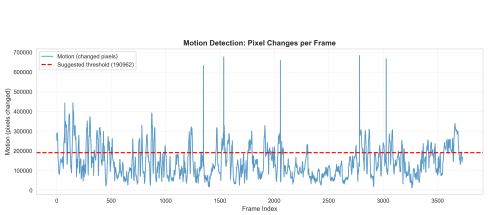


(d) Threshold sensitivity analysis over the CLIP similarity threshold.

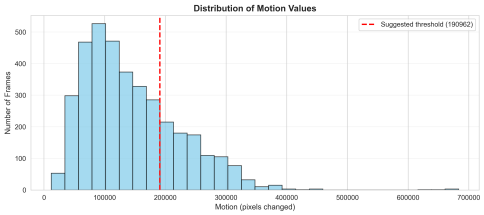
Fig. 2: Summary of main experiment plots (baseline comparison, heatmap, ablation, and threshold sensitivity) arranged compactly.

5 Visualizations

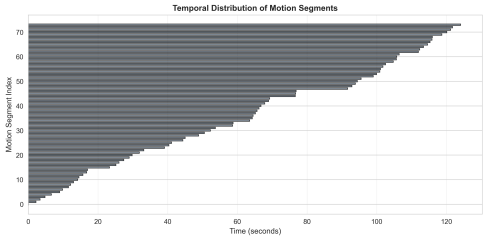
This section collects the plots generated by the experiment scripts. In addition to the aggregate baseline/ablation/threshold figures, we include diagnostic distributions and timeline plots that help interpret a single run.



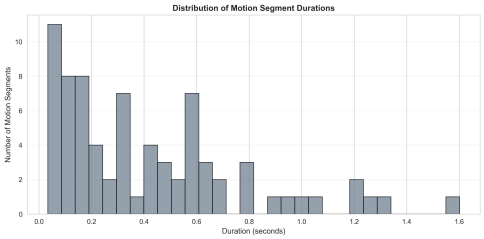
(a) Motion over time



(b) Motion distribution

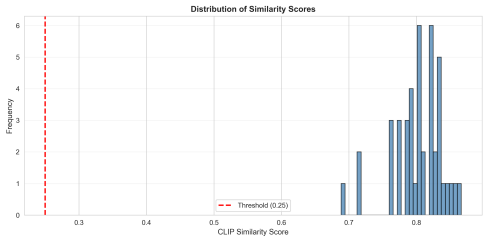


(c) Motion segments timeline

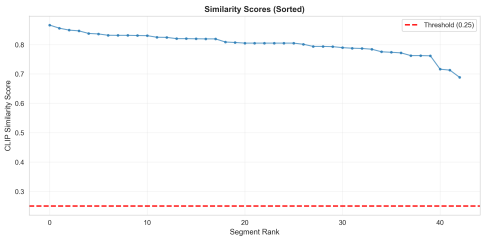


(d) Motion segment durations

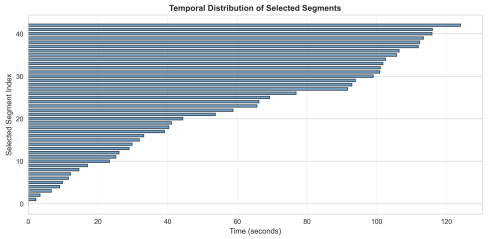
Fig. 3: Diagnostics for motion extraction. Top: per-frame motion values and their distribution. Bottom: temporal density of detected motion segments and their duration distribution.



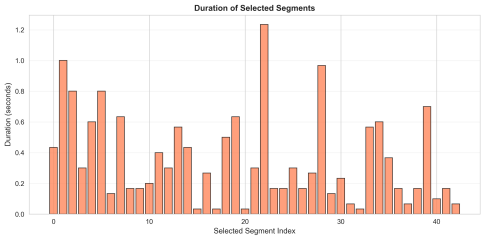
(a) Similarity distribution



(b) Similarity scores (sorted)



(c) Selected segments in time



(d) Selected segment durations

Fig. 4: Diagnostics for a single proposed-method run. Top: CLIP similarity score distribution and ranking (with threshold). Bottom: where selected segments occur in time and their durations.

6 Conclusion

We presented a modular pipeline for video montage generation guided by textual prompts. The method combines motion based candidate selection with caption generation and semantic matching. Experiments show that the proposed method outperforms simple baselines, semantic filtering improves efficiency and selectivity, and the similarity threshold can be robust in stable score regimes.

Disclosure Statement

This project was developed for an academic course assignment. The author declares no competing interests.

References

[1] A Radford et al. *Learning Transferable Visual Models From Natural Language Supervision*. ICML 2021.

[2] J Li et al. *BLIP Bootstrapping Language Image Pre training for Unified Vision Language Understanding and Generation*. CVPR 2022.

[3] N Reimers and I Gurevych. *Sentence BERT Sentence Embeddings using Siamese BERT Networks*. EMNLP 2019.

[4] M Honnibal et al. *spaCy Industrial Strength Natural Language Processing in Python*. Software documentation 2020.

[5] Zulko. *MoviePy Video Editing with Python*. Software documentation.